

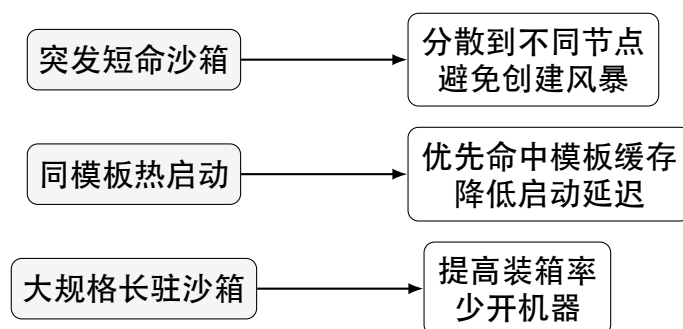
CubeSandbox 调度插件系统方案

摘要

AI Agent 的沙箱负载差异大，突发短命、同模板热启动、大规格长驻需要不同的调度策略，一套固定策略不够用。本方案在不修改现有接口的情况下，做三件事：把插件改成按名字登记，把调度策略打包成一份可切换的配置，用压测数据对比各策略效果。

问题

调度器的矛盾是需求多变、现状固定。
负载这边，三类典型场景的要求互相冲突：



三种要求方向相反，没有一套固定策略能同时满足。
现状这边，调度器恰好只有一套固定策略，还有三个缺口：

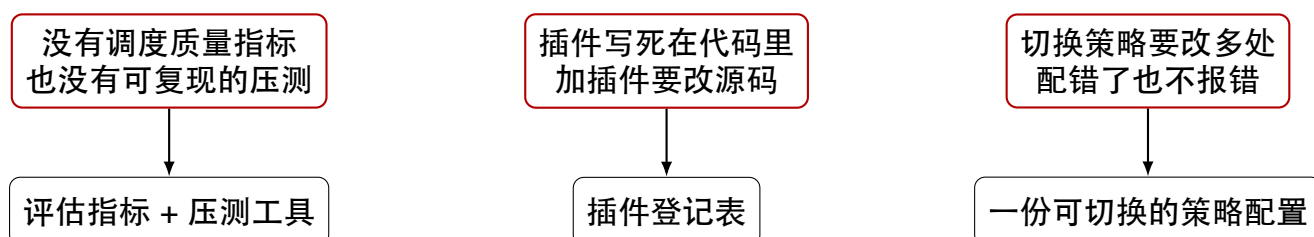
没有调度质量指标
也没有可复现的压测

插件写死在代码里
加插件要改源码

切换策略要改多处
配错了也不报错

方案总览

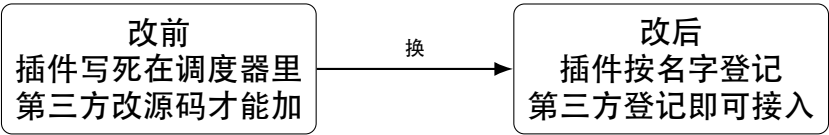
三件事一一对应地填上三个缺口：



方案细节

登记表取代写死的代码

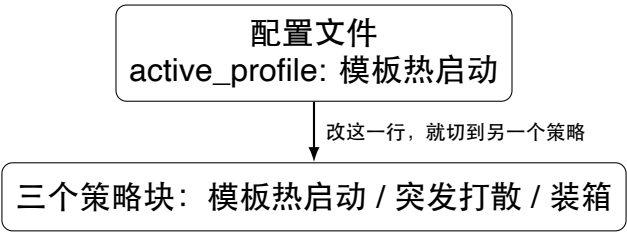
现在的插件写死在调度器源码里，第三方想加插件得改源码。改成一张插件登记表，插件启动时按名字登记，调度时按名字查表：



好处：插件名写错，启动即报错；构造失败返回错误；第三方登记后重新编译即可接入。

一份配置打包一套策略

把过滤、打分、权重、选点打包成一个有名字的策略块，切换策略只改一个名字、重启即生效：



每个插件的参数放在自己的参数块里，加新插件只在配置里加一段。没有写新策略时，程序把旧的几处配置自动翻译成默认策略，旧配置照常生效。

内置策略

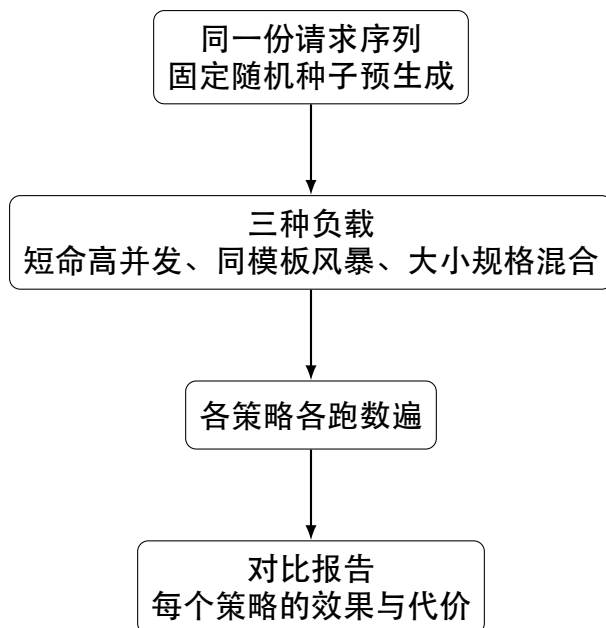
三套策略由上面的配置组合而成，复用现有插件，仅在现有插件不够时新写：

策略	适用场景	思路	代价
burst-spread	突发秒级沙箱	摊到最闲节点，避免创建风暴	缓存命中率下降
template-hotstart	同模板热启动	优先选已缓存模板的节点	缓存节点压力更大
binpack	大规格长驻沙箱	紧凑装箱，少开机器少碎片	负载更不均，故障面更大

每个策略改善一个指标的同时牺牲另一个，报告同时列出两方面。

指标与压测

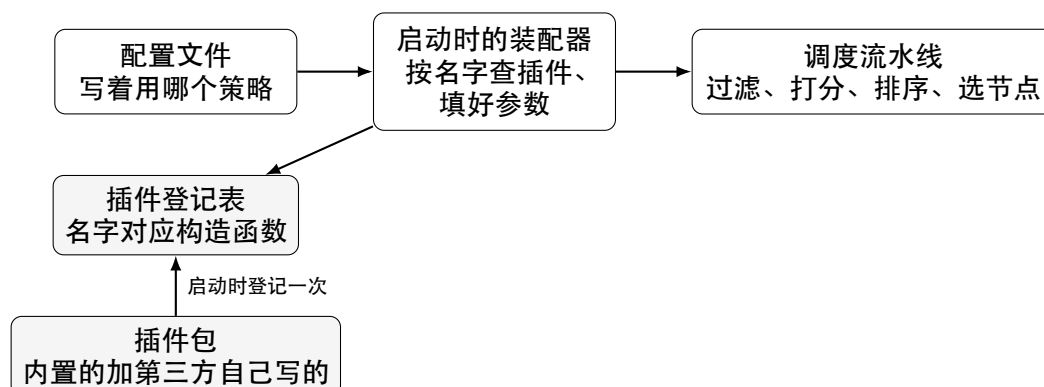
补上调度质量指标，实测和仿真两条路都做，对比各策略：



两条路各测一类指标，结果都进报告：

- **cube-bench** 真机：真实创建沙箱、走完整链路，测创建延迟和成功率；
- 仿真器：模拟节点与调度过程，评估装箱率、碎片率、负载均衡度，支持大规模集群。

整体装配流程：



自定义插件

提供两个示例插件：

- **label_filter**，过滤插件：只留下带指定标签的节点；
 - **zone_spread_score**，打分插件：把沙箱分散到不同可用区。
- 两个例子分别演示过滤和打分，写完登记、重新编译就能用，配套中英文文档。

实施细节

登记表

filter 包和 score 包各新增一个 registry.go，接口如下：

```
type Factory func(args json.RawMessage) (Selector, error)

func Register(name string, f Factory) error
func New(name string, args json.RawMessage) (Selector, error)
func Names() []string // 排序返回, 便于诊断
```

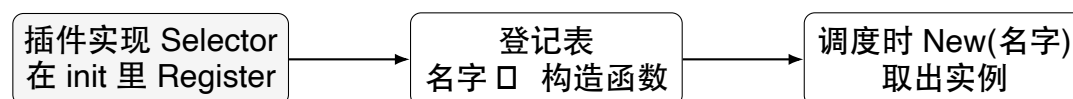
内置插件在各自 init 里登记, 登记名沿用现有配置取值。以 filter 包为例:

```
func init() {
    Register("cpu", func(args json.RawMessage) (Selector, error) {
        return NewCpuFilter(), nil
    })
    Register("mem", func(args json.RawMessage) (Selector, error) {
        return NewMemFilter(), nil
    })
    // template_locality、realtime_create_num、disk、thirtparty 同理
}
```

全部内置插件的登记名:

类型	数量	登记名
filter	6 个	cpu、mem、template_locality、 realtime_create_num、disk、thirtparty
score	4 个	real_time_weighted_average、 multi_factor_weighted_average、 affinity_score、image_score

登记之后, 插件与调度之间的调用关系:



改动清单:

- 移除 filters / scores 两个 map, 去掉 reflect.ValueOf 查表逻辑;
- 四处构造 panic 改为返回 error;
- 注册表用读写锁保证并发安全。

策略配置

新增 profiles 配置, 结构如下:

```
scheduler:
  active_profile: template-hotstart
  profiles:
    - name: template-hotstart
      filters: [{name: cpu}, {name: mem}, {name: disk}]
      scores:
        - name: image_score
          weight: 4
          args: { factors: { template_id: 1 } }
        - name: real_time_weighted_average
          weight: 1
          args: { factors: { realtime_create_num: 3 } }
      picker: { mode: best }
```

启动时按 `active_profile` 装配，对每个插件依次做四步：

1. 按名字查登记表；
2. 解码插件的 `args`；
3. 校验参数；
4. 实例化插件。

四步产出过滤器和打分器两组插件，交给现有调度流程，任何一步失败直接启动报错。插件的 `weight` 字段在实例化时注入，`Weight()` 返回该值。

`profiles` 留空时，把现有的 `enable_filters`、`enable_scorers`、`plugin_conf`、`resource_weights` 翻译成默认策略。其中 `picker` 由两个老配置联合翻译：

老配置 (<code>least_select_name</code> + <code>priority_select_num</code>)	等价 <code>picker</code>
<code>random + -1</code> (缺省组合)	<code>top_n_uniform</code> , <code>n=</code> 全部候选
<code>random + N</code>	<code>top_n_uniform</code> , <code>n=N</code>
<code>rw + N</code>	<code>top_n_weighted</code> , <code>n=N</code>
<code>sw / rrw + 任意</code>	<code>best</code>

`picker` 三种模式的语义：

- `best`：选得分最高的节点；
- `top_n_uniform`：在得分前 `N` 名中等概率随机；
- `top_n_weighted`：在前 `N` 名中按分数加权随机。
`n` 缺省或为 `-1` 表示全部。

选点确定化

改动在 `node.go` 的 `AllSortByScore()`。现在它用不稳定的 `sort.Slice` 且只按分数降序，输入来自 `map` 遍历，同分时结果随机。改成：

```
sort.Slice(l, func(i, j int) bool {
    if l[i].Score != l[j].Score {
        return l[i].Score > l[j].Score
    }
    return l[i].ID() < l[j].ID() // 同分按节点编号
})
```

同样的输入永远选出同样的节点，这是后面压测对比能复现的前提。

指标

在 `metrics.go` 集中定义，走现有 Prometheus 链路：

指标	类型	含义
scheduler_attempts_total{profile,result,reason}	Counter	调度尝试数，result 为成功/失败
scheduler_duration_seconds{profile}	Histogram	单次调度耗时，算 P50/P95
scheduler_decisions_total{profile,template_hit}	Counter	成功决策及模板命中数
scheduler_reschedules_total{profile,reason}	Counter	重调度次数
sandbox_create_attempts_total{profile,result}	Counter	创建成功/失败次数
sandbox_create_duration_seconds{profile,result}	Histogram	受理到建成的端到端耗时
scheduler_cluster_quota{resource,type}	Gauge	集群分配量与超卖后容量
scheduler_node_load_cv{resource}	Gauge	节点负载变异系数，衡量均衡度
scheduler_active_nodes / scheduler_empty_nodes	Gauge	活跃节点数 / 空节点数
scheduler_fragmented_capacity_ratio{shape}	Gauge	碎片率

标签固定用策略名和结果，控制 Prometheus 时序规模。速率和分位数由 counter/histogram 在 PromQL 里推导。

压测

扩展 cube-bench，三种负载的默认参数：

负载	参数	模拟目标
burst	500 请求、泊松 50/s、寿命 U(10s,120s)、小规格	高并发短命
template_storm	300 请求、同一 TemplateID、30% 节点预置	同模板风暴
mixed_spec	400 请求、1C2G:2C4G:8C16G=6:3:1、混合寿命	大小规格混合

请求序列按固定随机种子预生成，同一份序列给所有策略用，保证对比公平。